Day 18 – Spring Framework Notes

Record Class vs Lombok | Introduction to Annotations

1. Record Class vs Lombok

Before using the **record** class, first check which Java version the project is using. The record class is available from **JDK 14** onward. If the project uses an older JDK, the version needs to be updated.

How to update the Java version in pom.xml:

Change this:

```
<properties>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

to this:

```
<properties>
<maven.compiler.release>17</maven.compiler.release>
</properties>
```

After making this change, update the Maven project. This makes the record feature (and other features up to JDK 17) available for use.

Now let us compare the record class with Lombok:

1. Records are immutable

In a record class, once an object is created, its values cannot be changed. If a change is needed, a new object has to be created. A record does not generate setter methods, whereas Lombok's **@Data** (or @Setter) generates them, so its objects can be modified after creation.

```
public record Student1(
String name,
int id,
String address,
String adharCard) {
}

// With record
Student1 s = new Student1("abc", 101, "hyd", "12312312");
System.out.println(s);
s.address("banglore"); // Not allowed - no setter exists

// With Lombok
Student s1 = new Student("abc", 102, "hyd", "12312312", "xyz");
```

```
System.out.println(s1);
s1.setAddress("banglore"); // Allowed
```

2. Setter injection is not possible with a record

Since a record does not have any setter methods, Spring cannot perform setter injection on it through XML configuration.

```
public record Student1(
String name,
int id,
String address,
String adharCard) {
}
```

application-context.xml

```
<bean id="student1" class="com.day17.Student1">
<property name="name" value="prashant"></property>
</bean>
```

This configuration throws an exception, because setter injection is written for a **name** property, but no corresponding setter method exists in the record.

3. A record cannot extend or reuse another class

A record class cannot extend any other class. This means inheritance cannot be used with records.

```
public class University {
}

public record Student1(
String name,
int id,
String address,
String adharCard) extends University { // Not allowed
}
```

2. Introduction to Annotations

Until now, Spring Core has been learned using the XML approach.

```
<bean id="emp" class="com.demo.Employee"/>
```

Now imagine a project that contains 200 classes. Writing 200 separate bean configurations inside an XML file becomes a real problem.

Problems with the XML approach:

- The XML file becomes very lengthy.
- Configuration is stored separately from the Java class it belongs to.
- More typing is required.
- There are more chances of making mistakes.
- The file becomes difficult to maintain over time.

To solve these problems, Spring provides **annotation-based configuration**. But before learning Spring's annotations, let us first understand what an annotation actually is.

What is an Annotation?

An annotation is a special keyword that starts with the **@** symbol. It provides additional information (metadata) about a class, method, field, or constructor. It can be written at several places in a class:

- Class level
- Method level
- Field level
- Constructor level
- Method parameter level

```
@Override
public String toString() {
return "Employee";
}
```

Does @Override change the method's implementation?
No. It only gives extra information to the compiler. In the same way, Spring also uses annotations to provide configuration information.

2.1 Annotations are short and easy to memorize

Annotations are small keywords, such as **@Override**, **@Deprecated**, and **@SuppressWarnings**. Every annotation is just one short word starting with @. Instead of writing lengthy XML configuration, a developer only needs to remember a simple keyword.

Real-life example
Think about WhatsApp. Instead of typing “I am happy,” we simply send a smiley. The emoji is small, but it conveys the complete meaning. In the same way, @Override is short, but it carries useful information.

KEY POINT
✓ Annotations are short in nature, so they are easy to remember.

2.2 Annotation attributes can be written in any order

Some annotations contain attributes. Let us create a custom annotation of our own:

```
package com.day18;
```

```
public @interface Student {
String name();
int id();
}
```

```
package com.day18;

@Student(name = "prashant", id = 101)
public class University {
}
```

```
package com.day18;

import java.lang.annotation.Annotation;

public class Main {
public static void main(String[] args) {

// In this way we cannot read the content of the annotation
University uni = new University();
System.out.println(uni);

// This is the way we CAN read the annotation
Class<University> ref = University.class;
Student annotation = ref.getAnnotation(Student.class);

System.out.println("ID : " + annotation.id());
System.out.println("Name: " + annotation.name());
}
}
```

By default, a custom annotation has the retention policy **CLASS**. This means it is stored in the .class file, but it is not available while the program is running. Because of this, the line **ref.getAnnotation(Student.class)** returns **null**, and reading annotation.id() throws a NullPointerException.

Annotation visibility – the three types

1. Source code annotations – These appear in the source code but are not retained during compilation or at runtime. Their only use is as a documentation helper.

```
@Author(name = "Prashant")
class Feature {
// logic to perform the operation
}
```

2. Compile-time annotations – The Java compiler reads these annotations at compile time and applies some validation, or includes additional bytecode based on them.

```
class Animal {
public void sound() {
System.out.println("Animal Sound");
}
}

class Dog extends Animal {

@Override // The compiler checks this at compile time
public void sound() {
System.out.println("Dog Bark");
}
}
```

3. Runtime annotations – These are included in the bytecode of the generated class file and can be read at runtime. Custom exceptions are a good example of this. To make a custom annotation available at runtime, add **@Retention(RetentionPolicy.RUNTIME)** to it.

```
package com.day18;

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.RUNTIME)
public @interface Student {
String name();
int id();
}
```

Output: `Name: Prashant, id=101`

Both of the following are correct, because Java identifies attributes by their names, not by their position:

```
@Student(id = 101, name = "Prashant")

// or

@Student(name = "Prashant", id = 101)
```

Real-life example
Suppose someone asks for your details:
Name: Prashant, Age: 28-30
or
Age: 28-30, Name: Prashant
The information is the same either way, because every value carries its own label. In the same way, annotation attributes can be written in any order.

KEY POINT

✓ Annotation attributes are flexible.

✓ There is no need to remember any specific order.

2.3 Annotations are a Java language feature

Annotations were not created only for Spring – they are already part of the Java language. Examples include @Override, @Deprecated, and @SuppressWarnings. Later, Spring introduced its own annotations, such as @Component, @Autowired, and @Service. Notice that the syntax always stays the same (starting with @); only the annotation name changes.

Real-life example
Driving a bike and driving a scooter – the controls are almost the same, and only the vehicle changes. In the same way, developers who already know how annotations work only need to learn the new annotation names that Spring introduces.

KEY POINT

✓ Java developers do not need to learn a new syntax – they only learn new annotation names.

2.4 Annotations participate in Java compilation

When Java compiles code, it also reads the annotations present in it.

```
@Override
public String toString() {

}
```

If there is no toString() method in the parent class, the compiler immediately raises an error, because it reads the @Override annotation during compilation.

```
@Deprecated
public void oldMethod() {

}
```

Whenever another developer uses oldMethod(), the IDE shows a warning. This warning is possible only because annotations are available during compilation.

KEY POINT

✓ Annotations participate in the Java compilation process.

2.5 Annotations are written directly on the source code

Annotations are written exactly where they are needed.

```
@Override
public void display() {

}
```

Since the annotation is written directly above the method, its purpose becomes clear while reading the code itself.

> **Real-life example**
> Imagine a class notebook. Case 1: notes are written on the same page as the chapter, so they are easy to understand. Case 2: notes are written in a separate notebook, so they need to be searched for every time. Annotations work like Case 1 – they keep information together with the code.

> **KEY POINT**
> ✓ Configuration stays close to the source code.
> ✓ The code becomes easier to read and understand.

2.6 All Java IDEs support annotations

Whether the IDE is Eclipse, IntelliJ IDEA, VS Code, or NetBeans, every Java IDE understands annotations. Typing @ makes the IDE immediately suggest @Override, @Deprecated, and @SuppressWarnings. Typing something incorrect, such as @Overridee, makes the IDE show an error right away.

> **Real-life example**
> Google predicts the next word while typing a search query. In the same way, an IDE predicts annotation names and helps write them correctly.

> **KEY POINT**
> ✓ Auto-completion.
> ✓ Error highlighting.
> ✓ A better overall coding experience.

2.7 Annotations support RAD (Rapid Application Development)

RAD stands for Rapid Application Development, which means developing applications faster. Annotations make this possible because they reduce XML configuration, typing, boilerplate code, and maintenance effort. Less work leads to fewer mistakes, which leads to faster development.

> **Real-life example**
> Earlier, making a payment meant going to the bank, filling a form, standing in a queue, and then depositing money. Now, it only takes opening a UPI app, scanning a QR code, and the payment is done. The work is the same, but it happens much faster. Annotations reduce unnecessary configuration work in the same way, which is why they support Rapid Application Development.

Looking ahead: Now that the concept of an annotation in Java is clear, the next step is to see how Spring uses this same idea. Instead of writing configuration inside XML files, Spring allows configuration to be written directly inside Java classes using small annotations. The next topic covers the first Spring annotation.